Google Summer of Code 2026
SOFIE Project Exercise Report

ML Inference on Heterogeneous Architectures using SOFIE and Alpaka

Candidate

Nisma Amjad

GSoC 2026 Prospective Contributor

https://github.com/Nismamjad1/SOFIE

Mentors

Lorenzo Moneta
Sanjiban Sengupta

CERN / ROOT Team

Project:	ML Inference on Heterogeneous Architectures
Organisation:	CERN-HSF / ROOT
Branch:	`gsoc2026-nisma`
Submission:	14 March 2026
Hardware:	Dell Precision 7680, NVIDIA RTX 5000 Ada, CUDA 12.2
Platform:	Ubuntu 24.04

Contents

1 Exercise 1

1.1 Conceptual Understanding

ROOT:
A data analysis framework developed by CERN, used for storing, processing, and analysing large-scale particle physics data.

TMVA (Toolkit for Multivariate Analysis):
A machine learning library built into ROOT that allows physicists to train and evaluate ML models.

PyMVA:
A Python bridge inside TMVA that connects it to external deep learning frameworks like Keras and PyTorch.

SOFIE:
A TMVA module that takes a trained model (from ONNX, Keras, or PyTorch) and generates a standalone, lightweight C++ header file for fast inference.

Alpaka:
A C++ library for hardware-portable parallel computing. A kernel written once in alpaka can run on NVIDIA GPUs (CUDA), AMD GPUs (HIP), or CPU without any code changes.

1.2 Building ROOT from Source

ROOT was built from source on Ubuntu 24.04 (Dell Precision 7680, NVIDIA RTX 5000 Ada, CUDA 12.2) with the following configuration:

- TMVA with SOFIE support (`-Dtmvasofie=ON`)
- PyMVA enabled with Python, NumPy, TensorFlow, and PyTorch
- Google Protobuf installed as a dependency
- VDT installed

Building ROOT from source with SOFIE support required resolving several configuration issues, including disabling `runtime_cxxmodules` (`-Druntime_cxxmodules=OFF`). For the SOFIE standalone repository, two additional issues arose: Protobuf and VDT had to be explicitly pointed to via CMake arguments (`-DVDT_INCLUDE_DIR`, `-DVDT_LIBRARY`) since they were not found automatically, and the CUDA toolkit was not in `PATH` by default, requiring the following to be set before CMake could detect `nvcc` and enable the alpaka CUDA backend:

```
export PATH=/usr/local/cuda-12.2/bin:$PATH
```

The final working CMake command is shown in Figure 1.

The Final Working cmake Command

After all iterations, what actually worked:

bash

```
cmake /home/nisma/root_src \
  -DCMAKE_INSTALL_PREFIX=/home/nisma/root_install \
  -Dtmva-sofie=ON \
  -Dtmva=ON \
  -Dtmva-pymva=ON \
  -Dpython3=ON \
  -DPYTHON_EXECUTABLE=/home/nisma/envs/gsoc-root/bin/python3 \
  -Dfitsio=OFF \
  -Druntime_cxxmodules=OFF
```

Figure 1: The final working CMake configuration command for building ROOT with SOFIE and alpaka CUDA support.

2 Exercise 2

The `TMVA_Higgs_Classification` and `TMVA_CNN_Classification` tutorials were run, observing the full training and evaluation pipeline.

2.1 TMVA_Higgs_Classification.C

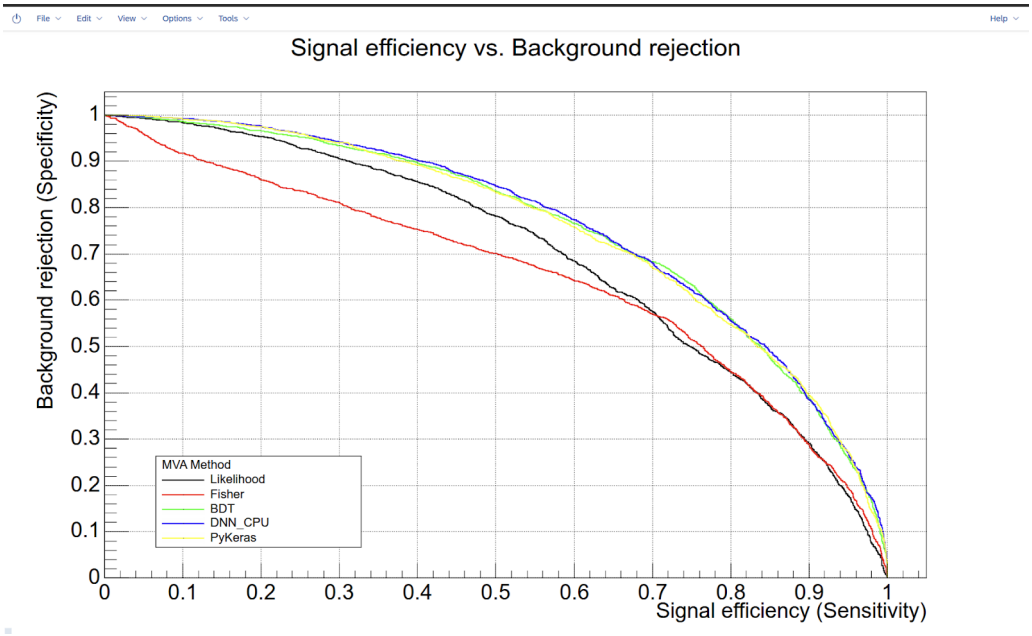

Figure 2: Signal efficiency vs. Background rejection for `TMVA_Higgs_Classification`.

PyKeras and DNN_CPU curves overlap almost perfectly, confirming that PyMVA successfully bridges Keras to TMVA and produces results equivalent to TMVA's native DNN, validating that the `pymva` interface works.

2.2 TMVA_CNN_Classification.C

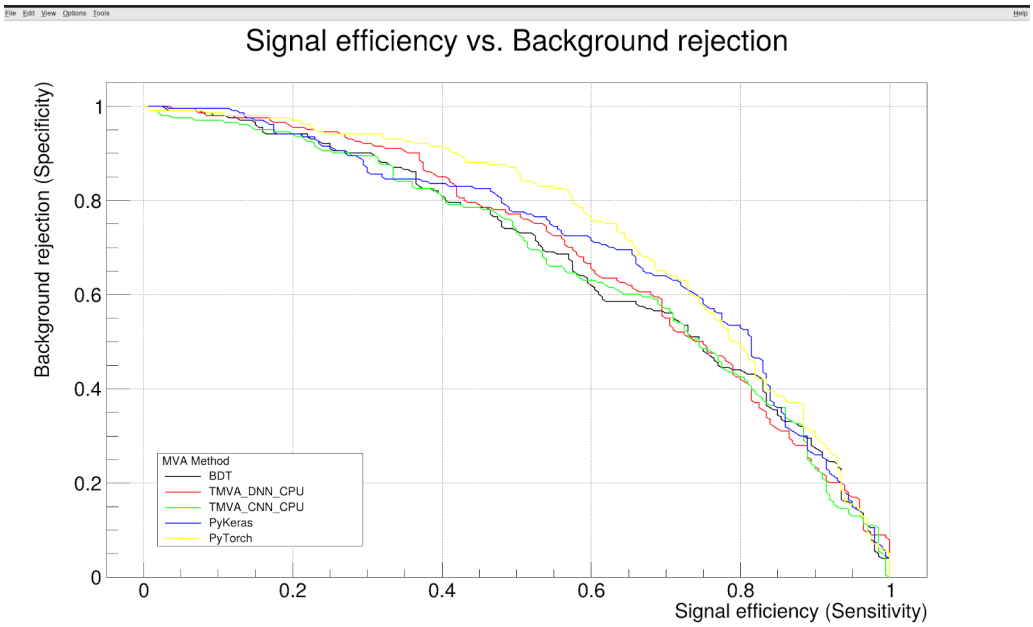

Figure 3: Signal efficiency vs. Background rejection for `TMVA_CNN_Classification`.

Key Observation

The PyTorch and Keras models outperform TMVA's native CNN, bringing better-optimised implementations into the ROOT ecosystem. Once you have a better Keras/PyTorch model, you want to deploy it efficiently, which is exactly what SOFIE enables.

2.3 SOFIE Tutorials

The PyMVA module was explored to understand how it bridges TMVA to the Keras and PyTorch backends. The SOFIE tutorials (`TMVA_SOFIE_ONNX.C`, `TMVA_SOFIE_Keras.C`, `TMVA_SOFIE_PyTorch.C`) were run, which gave a clear picture of the ONNX parsing pipeline and how SOFIE generates standalone C++ inference code from trained models.

Overall Observation

Across all three tutorials, SOFIE produced structurally identical generated code regardless of the source framework, confirming that it provides a unified, framework-agnostic inference path from any trained model to production C++ code.

3 Exercise 3

The `ML4EP/SOFIE` repository was cloned and the `gpu/alpaka` branch was checked out. After resolving build issues, the project was built successfully with CUDA 12.2 and compute capability 8.9.

3.1 Architecture Observations

- Each operator that supports GPU inference implements three methods: `Generate_GPU_Kernel_AI` (writes the kernel struct), `Generate_GPU_Kernel_Definitions_ALPAKA()` (declares the kernel instance inside the Session), and `Generate_GPU_ALPAKA()` (emits the `alpaka::exec()` launch call).
- `RModel_ALPAKA.cxx` drives code generation, iterating over operators and calling these methods to produce a self-contained `.hxx` inference file.
- The generated `Session` struct handles device/queue setup, GPU buffer allocation, host-to-device transfers, and kernel dispatch.

3.2 Gaps Identified

- `Softmax` had no GPU implementation despite being listed in `single_initialized_operators`.
- `BatchNormalization`, `Conv`, `ConvTranspose`, and `Where` were also missing GPU support.
- The `Where` operator was explicitly skipped in tests due to an unsupported Boolean tensor type.
- No benchmark or performance comparison infrastructure existed.
- ROCm/HIP backend was untested.

4 Exercise 4

The Softmax operator was listed in SOFIE's single_initialized_operators set, but had no actual GPU implementation, calling it would silently fall back to CPU. The goal was to implement full GPU inference support for Softmax using the alpaka heterogeneous computing library.

4.1 Implementation

Three methods were added to `ROperator_Softmax.hxx`:

1. **`Generate_GPU_Kernel_ALPAKA():`** generates an alpaka kernel struct where each GPU thread handles one complete softmax vector using a numerically stable 3-pass algorithm: find the maximum value, compute $\exp(x - \max)$ for each element, then divide by the sum. Subtracting the max prevents floating-point overflow.

2. **`Generate_GPU_Kernel_Definitions_ALPAKA():`** declares the kernel instance inside the generated `Session` struct.

3. **`Generate_GPU_ALPAKA():`** emits the `alpaka::exec()` kernel launch call with the correct `axis`, `stride`, and `numVectors` parameters computed from the tensor shape, supporting arbitrary tensor rank and softmax axis.

`RModel_ALPAKA.cxx` was also updated to register `SOFTMAX` in the `single_initialized_operators` set.

Test Results

Two GTest unit tests were added to `TestCustomModelsFromONNXForAlpakaCuda.cxx`, **Softmax1d** and **Softmax2d** , which run the kernel on an RTX 5000 Ada GPU and compare output against numpy-computed reference values within a tolerance of 1e-4. Both passed. All 11 tests in the test suite passed with 0 failures.